

SISTEMA DE COMÉRCIO ELETRÔNICO DIDÁTICO — IMPLEMENTAÇÃO COM PHP, PDO E MYSQL

Grupo Projeto Final

Andreivy M.Rodrigues | Vivian Belo da Silva | Leonardo Bruno Ferreira de Almeida

Face Wyden / Curso de Ciências da Computação
Belém/PA — 2025

Resumo

Este artigo descreve o projeto, a implementação e a avaliação de um sistema de comércio eletrônico didático desenvolvido com tecnologias web tradicionais: HTML5, CSS3, JavaScript e PHP com integração ao banco de dados MySQL via PDO. O sistema implementa funcionalidades essenciais de um e-commerce, incluindo cadastro e gerenciamento de produtos (CRUD), catálogo dinâmico, carrinho de compras baseado em sessão, finalização

de pedido, e persistência transacional dos pedidos. A abordagem privilegia boas práticas de segurança, como uso de prepared statements e transações, e busca demonstrar a integração entre camadas front-end e back-end de forma responsiva e navegável.

Palavras-chave: comércio eletrônico; PHP; PDO; MySQL; CRUD; segurança; ABNT.

1 Introdução

O crescimento do comércio eletrônico exige sistemas confiáveis, seguros e fáceis de manter. Este trabalho apresenta o desenvolvimento de um protótipo de loja virtual com foco didático, cujo objetivo principal é consolidar conhecimentos sobre integração entre aplicações web e um Sistema de Gerenciamento de Banco de Dados (SGBD). A implementação destaca a utilização da classe PDO do PHP para acesso ao banco MySQL, práticas de segurança (prepared statements), e a composição de um fluxo completo de compra (do cadastro de produtos à gravação de pedidos).

1.1 Objetivos

- Projetar e implementar um sistema de e-commerce funcional, com persistência de dados e operações CRUD para produtos.
- Garantir a segurança das operações de banco (prevenção de SQL injection) e a integridade dos dados via transações.
- Demonstrar a integração entre frontend (HTML/CSS/JS) e backend (PHP/PDO), com interfaces responsivas.

2 Desenvolvimento — Metodologia e Tecnologias

2.1 Metodologia

A metodologia adotada foi incremental e orientada por funcionalidades: inicialmente modelou-se o banco de dados relacional com tabelas para produtos, pedidos e itens de pedido; em seguida implementou-se camadas de acesso a dados (DAO simples via PDO), endpoints para operações do carrinho e controle administrativo (CRUD de produtos), além das interfaces HTML/CSS/JS para interação do usuário. O teste foi realizado manualmente em ambiente local (XAMPP/LAMP) com dados de amostra.

2.2 Tecnologias utilizadas

- PHP (≥ 7.4): linguagem de servidor, gerenciamento de sessões e execução lógica do negócio.
- PDO (PHP Data Objects): camada de acesso a dados com suporte a prepared statements e tratamento de exceções.
- MySQL/MariaDB: SGBD relacional para persistência de produtos, pedidos e itens de pedidos.

- HTML5, CSS3: marcação semântica e estilo responsivo conforme práticas modernas.
- JavaScript: comportamento no lado do cliente — atualização do carrinho, navegação e pequena UX.

2.3 Estrutura do banco de dados

O esquema contém as tabelas principais: products (id, sku, nome, descricao, preco, imagem, created_at); orders (id, nome_cliente, email, endereco, total, status, created_at) e order_items (id, order_id, product_id, nome, quantidade, preco). O DB foi criado conforme script SQL (arquivo data/db.sql) e os dados iniciais foram semeados a partir de um arquivo JSON (data/products.json).

2.4 Boas práticas de segurança e arquitetura

- Uso de prepared statements (PDO::prepare) para todas as consultas que recebem parâmetros do usuário, prevenindo SQL injection.
- Configuração de PDO para lançar exceções (ERRMODE_EXCEPTION) e retorno em modo associativo (FETCH_ASSOC).
- Utilização de transações para o registro de pedidos e itens relacionados, garantindo atomicidade.
- Separação simples entre templates (layouts) e lógica; em versão futura recomenda-se introduzir camada de serviços e autenticação.

3 Descrição das páginas e funcionalidades

3.1 Catálogo de Produtos (index.php)

Página dinâmica que exibe produtos do banco de dados. Para cada item, o usuário pode escolher quantidade e adicionar ao carrinho. A listagem é efetuada por consulta SELECT com PDO.

3.2 Carrinho (carrinho.php + php/cart_actions.php)

Estrutura de carrinho em sessão; operações possíveis: adicionar, atualizar quantidade, remover. cart_actions.php recebe ações via POST e manipula \$_SESSION['cart'].

3.3 Checkout (checkout.php, php/place_order.php)

Ao confirmar o pedido, o servidor grava o cabeçalho do pedido em orders e, dentro de uma transação, persiste cada item em order_items. Após commit, o carrinho de sessão é limpo.

3.4 Área Administrativa (php/admin/*)

Interface para gerenciamento de produtos, com os formulários de

criação e edição, e a listagem para exclusão. Todas as entradas administrativas são validadas e persistidas com prepared statements.

4 Resultados — Demonstração do sistema em funcionamento

4.1 Cenário demonstrativo

- Inicialização: script `php/init_db.php` cria o schema e popula produtos de exemplo.
- Fluxo do usuário: navegar no catálogo (`index.php`), adicionar produtos ao carrinho, alterar quantidades em `carrinho.php`, finalizar compra em `checkout.php` e observar a gravação em `orders/order_items`.
- Administração: criar/editar/excluir produtos via `php/admin/products_*` e verificar impacto imediato no catálogo.

A implementação permitiu compreender na prática a integração entre front-end e back-end e a responsabilidade do SGBD em garantir persistência e integridade. O uso de PDO e prepared statements mostrou-se simples e eficaz para mitigar riscos de injeção SQL. A experiência também evidenciou pontos de melhoria: implementação de autenticação e controle de acesso, proteção CSRF para operações por formulário, validação de imagens e uploads seguros, além de testes automatizados.

5.1 Perspectivas de melhoria

- Inserir autenticação por perfis (administrador / cliente) e controle de permissões.
- Implementar proteção CSRF em formulários.
- Implementar upload e armazenamento seguro de imagens (server-side validation, sanitização e armazenamento em blob ou object storage).
- Adicionar testes automatizados (PHPUnit para backend; Playwright/Cypress para testes end-to-end).

Referências

1. ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724:2011 — Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro: ABNT, 2011.
2. PHP: THE RIGHT WAY. PHP: The Right Way. Disponível em: <<https://phptherightway.com/>>. Acesso em: 27 nov. 2025.
3. PHP MANUAL. PDO — PHP Data Objects. Disponível em: <https://www.php.net/manual/pt_BR/book.pdo.php>. Acesso em: 27 nov. 2025.
4. ORACLE CORPORATION. MySQL 8.0 Reference Manual. Disponível em: <<https://dev.mysql.com/doc/>>. Acesso em:

27 nov. 2025.

5. MOZILLA. MDN Web Docs — HTML, CSS e JavaScript. Disponível em: <<https://developer.mozilla.org/>>. Acesso em: 27 nov. 2025.